SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCENARIO BASED ASSESSMENT

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Scenario Based Assessment
Weightage:	20% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	S. MAGESH	**Reg. No.:**	192525002
Section / Batch:	B. TECH - AIML	**Date:**	21/07/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given scenario and identify the computational problem involved.
- Apply appropriate Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems wherever applicable.
- Clearly justify the chosen solution approach with proper reasoning and analytical interpretation.
- Demonstrate decision-making skills by comparing possible solutions and selecting the most suitable approach.
- Use diagrams, stepwise illustrations, or algorithmic representations wherever necessary.
- Maintain clarity, logical organization, and proper technical presentation throughout the answers.

Question Type	Focus Area	Questions
Scenario Based Assessment	Analyze real-world scenarios and apply Brute Force techniques for sorting, searching, Closest-Pair, and Convex-Hull problems	Q1

SECTION A – SCENARIO BASED ASSESSMENT

1. A university maintains student records where marks are almost sorted, except for a few recently updated entries. The system needs to sort the data frequently with minimal computation time. The developer is choosing between Selection Sort, Bubble Sort, and Insertion Sort.

a) Analyze all three algorithms in this context.
b) Recommend the most suitable one with justification based on comparisons and swaps.

A) ANALYSIS OF THE ALGORITHMS

The university stores student marks that are already almost sorted, with only a few recently updated records. In this situation, the sorting algorithm should make only a small number of changes.

- Selection Sort always searches the entire unsorted portion to find the minimum element. It performs the same number of comparisons even if the data is nearly sorted.
- Bubble Sort compares adjacent elements and swaps them when necessary. If an optimized version is used, it can stop early when no swaps occur.
- Insertion Sort inserts each updated record into its correct position. Since the data is almost sorted, only a few shifts are required.

B) RECOMMENDATION

Insertion Sort is the most suitable algorithm because it performs fewer comparisons and shifts when the dataset is nearly sorted. It is faster than Selection Sort and Bubble Sort in this scenario and requires less computation time.

CONCLUSION

For frequently updated student records that remain almost sorted, Insertion Sort provides the best performance because it efficiently handles small changes without repeatedly processing the entire dataset.

3. A hospital database stores patient records in an unsorted list because records are frequently added and updated. Doctors need to quickly search for patient details during emergencies.

a) Explain how sequential search works in this scenario.
b) Analyze best, worst, and average case performance.
c) Justify why sorting is not preferred here.

A) HOW LINEAR SEARCH WORKS

The hospital stores patient records in an unsorted list because new records are added regularly. During an emergency, the system starts from the first record and checks each patient's details one by one until the required record is found.

B) PERFORMANCE ANALYSIS

- Best Case: O(1), when the required patient record is the first entry.
- Average Case: O(n), when the record is somewhere in the middle.
- Worst Case: O(n), when the record is the last entry or does not exist.
-

C) WHY SORTING IS NOT PREFERRED

Sorting the database after every update would consume extra time because patient records are frequently added and modified. Keeping the data unsorted makes updates faster, even though searching may take longer.

CONCLUSION

Linear Search is a practical choice for frequently changing hospital records because it allows quick updates without the need to maintain sorted data.

5. A plagiarism detection system compares a pattern against a large document using brute-force string matching. The document length is very large, and the pattern appears rarely.

a) Explain the working of brute-force string matching.
b) Analyze its time complexity.
c) Discuss why performance degrades in this scenario.

A) WORKING OF THE ALGORITHM

The brute-force string matching algorithm compares the given pattern with every possible position in the document. If the characters match, it continues comparing until the entire pattern is matched. If a mismatch occurs, it shifts the pattern by one position and repeats the process.

B) TIME COMPLEXITY

- Best Case: O(n)
- Worst Case: O(n × m)

where:

- n = length of the document
- m = length of the pattern

C) WHY PERFORMANCE DEGRADES

When the document is very large and the pattern appears rarely, the algorithm performs many unnecessary comparisons before finding a match or reaching the end of the document. This increases the execution time.

CONCLUSION

Brute-force string matching is simple to implement but becomes inefficient for large documents because it repeatedly compares characters from many positions.

9. A dataset of student marks is stored in sorted order, and the system frequently needs to locate a specific mark using Binary Search. However, new records are inserted randomly, disturbing the sorted order.

a) Explain how Binary Search works when the data is sorted.
b) Analyze its time complexity in best, average, and worst cases.
c) Discuss why maintaining sorted order is critical and justify the impact on search performance.

A) HOW BINARY SEARCH WORKS

Binary Search works only when the data is arranged in sorted order. It checks the middle element first. If the required value is smaller, it searches the left half; otherwise, it searches the right half. This process continues until the element is found.

B) TIME COMPLEXITY

- Best Case: O(1)
- Average Case: O(log n)
- Worst Case: O(log n)

C) IMPORTANCE OF SORTED ORDER

If new records are inserted randomly, the sorted order is disturbed. Binary Search will no longer work correctly because it depends on the elements being arranged in ascending or descending order. The data must be sorted again before Binary Search can be used.

CONCLUSION

Binary Search is one of the fastest searching algorithms, but it is effective only when the dataset is maintained in sorted order.

10. A robotics system analyzes multiple obstacle points in a 2D plane to determine the boundary region using a brute-force approach. As the number of points increases, processing time becomes high.

a) Explain how brute-force geometric analysis (similar to convex hull) works.
b) Analyze the time complexity of checking all point combinations.
c) Discuss why this approach is not scalable and justify the need for efficient alternatives.

A) WORKING OF THE ALGORITHM

The robotics system checks different combinations of obstacle points to determine which points form the outer boundary. Every pair of points is examined to identify whether all other points lie on one side of the line joining them. The points that satisfy this condition become part of the convex hull.

B) TIME COMPLEXITY

The brute-force convex hull algorithm checks many point combinations, resulting in a time complexity of $O(n^3)$.

C) WHY IT IS NOT SCALABLE

As the number of obstacle points increases, the number of calculations grows rapidly. This causes longer processing time and higher computational cost, making the algorithm unsuitable for large datasets.

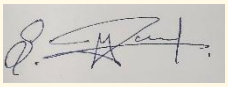
CONCLUSION

The brute-force convex hull method is easy to understand but becomes inefficient when handling a large number of points. More advanced algorithms are preferred for real-time robotic applications.

Code of Conduct

*I **S,MAGESH (192525002)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student:



RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30%	Applies appropriate concepts effectively to solve complex realworld problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & DecisionMaking Skills	20%	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15%	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10%	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25%				
Application of Concepts	30%				
Analytical & Decision-Making Skills	20%				
Solution Design & Approach	15%				
Clarity, Justification & Presentation	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____